Hands-On-Learning: Backend Reliability Challenge - Handle It Smart

Learning Objectives

By the end of this activity, learners will be able to:

- Implement a minimal LLM middleware with retry, backoff, and timeout handling.
- Detect rate-limit (429) and transient errors and respond with jittered backoff.
- Add lightweight request logging, including token usage and latency.
- Evaluate trade-offs between reliability, latency, and cost when designing a resilient backend.



Description

In this hands-on practice, you will build a resilient API-calling layer-similar to what real AI startups use to avoid outages during high user demand. You will implement retries with jittered backoff, add simple caching, and log per-request token usage. This gives you the baseline skills required to prevent cascading failures and create a stable LLM-backed application.

Structure of the Activity

Title: Backend Reliability Challenge - Handle It Smart

Dataset Information / Resources Required

This lab does not require a dataset. Instead, you will interact with an LLM endpoint using a minimal script.

Provided resources:

- Example request body (JSON)
- Example log format (logs/run_*.jsonl)

Tools Used

Primary language: Python 3.10+

Optional framework: FastAPI (if learners choose to wrap middleware in an API endpoint)

Libraries:

- openai (or compatible LLM SDK)
- httpx or requests (for direct HTTP calls if needed)

Optional: Postman or curl for manual testing

Setup Requirements

- Working Python environment
- Installed dependencies (pip install -r requirements.txt)
- Configured **OpenAI API key** (or equivalent model provider key)
- A logs/ directory for storing outputs
- Optional: FastAPI + uvicorn (pip install fastapi uvicorn)

Scenario

Your company builds a lightweight AI assistant used by several hundred customers daily. Recently, traffic increased and your app began failing unpredictably due to **API rate-limit errors** (HTTP 429). Users saw confusing timeouts and your logs showed bursts of failures without retries.

Your CTO asks you to build the **first version of a resilient middleware layer**:

“If we’re going to grow, our API calls can’t collapse under load. Build the guardrails so requests survive temporary failures.”

You must implement retries with jittered backoff, add minimal caching for deterministic calls, and log the details needed to analyze reliability.

Objective

By the end of this activity, you'll have a **working middleware script** that calls an LLM endpoint, handles rate limits gracefully, and logs token usage-plus a clear explanation of the retry logic and performance trade-offs.



Instructions for Participants

Getting Started

1. Open your IDE or code editor.
2. Create a project folder and a Python file such as middleware.py or use the provided starter.
3. Ensure your API key is loaded from environment variables or a config file.
4. Create a logs/ folder for storing JSONL logs.
5. Review the scenario and objectives.

coursera

Activity 1: Build the Resilient Middleware

Objective

Implement a simple LLM-calling function with retry, backoff, caching, and structured logging.



Input/Data

Inputs: A sample prompt string (you may hardcode or accept input)

Output: JSONL logs with timestamp, latency, token usage, retries, and cache hits

Steps to Complete

Step 1: Define Your Call Function

- Create a function (e.g., call_llm(prompt))
- Include parameters for model, temperature, and timeout

Step 2: Add Retry Logic with Jittered Backoff

- Implement a loop with:
  - Max retries (e.g., 3)
  - Exponential backoff (1s → 2s → 4s)
  - Random jitter added (e.g., ±200ms)

coursera

Retry on:

- HTTP 429 rate-limit errors
- 500-level transient failures
- Timeout exceptions

Step 3: Implement Basic Caching (Optional but Encouraged)

- Use an in-memory dictionary keyed by prompt text
- If prompt is already cached, return value instantly
- Log cache_hit=True when applicable

coursera

Step 4: Log Request Details

Each log entry should include:

- Timestamp
- Prompt ID or hash
- Total latency (ms)
- Retry count
- Token usage
- Cost estimate (if available)
- Cache hit flag

Step 5: Test Your Middleware

- Make a few manual calls using simple prompts
- Verify retry behavior by temporarily forcing exceptions
- Confirm logs are written to the logs/ folder

Expected Outcome

A Python script capable of calling an LLM endpoint, handling rate limits gracefully, retrying with jittered backoff, optionally caching responses, and creating structured logs.

Tips

- Keep retry and timeout settings small for testing.
- Use time.sleep() plus random jitter for backoff.
- Consider hashing the prompt for cleaner cache keys.

Activity 2: Evaluate Behavior Under Failure

Objective

Simulate or trigger rate-limit or timeout errors and observe how your middleware responds.



Input/Data

Middleware script from Activity 1

Any prompt of your choice

Steps to Complete

Step 1: Trigger Mild Load (Optional)

- Run 5–10 calls in quick succession (loop)
- Observe: Does retry fire? Does caching help?

Step 2: Manually Simulate Errors

- Temporarily modify your code to raise an exception on the first attempt
  - E.g., mock a 429 or timeout exception
- Confirm that the middleware retries with increasing backoff intervals
- Confirm logging reflects:
  - Retries
  - Latency
  - Final success or failure

Step 3: Verify Logging Detail

- Open the output JSONL and inspect each field
- Look for retry counts and token usage
- Confirm timestamps and latency look reasonable

Expected Outcome

You see your retry/backoff logic functioning in real time, you understand how the middleware defends against transient failures, and your logs reveal how many retries were needed.

Tips

- Use try/except blocks to capture API exceptions.
- Remember that **reliability often costs extra time**-that’s normal.

Compile Your Findings

1 Middleware Script

Requirement: Resilient middleware implementation plan.

Include:

- Retry/backoff logic
- Timeout handling
- Optional cache behavior
- Structured logging

2 Retry Policy Explanation

Requirement: A clear written summary of your retry strategy.

Include:

- Why you chose your number of retries
- Explanation of exponential + jittered backoff
- When retries stop and when the system fails fast
- Why this policy avoids cascading failures

3 Performance Reflection

Requirement: A thoughtful analysis of the trade-offs between latency, reliability, and cost.

Include:

- When retries help and when they hurt
- How caching changes latency and cost dynamics
- Why p95 behavior matters more than p50
- How middleware design impacts real user experience

Summary

This activity provides hands-on experience with building resilient LLM middleware through practical implementation of retry logic, backoff strategies, timeout handling, and lightweight observability. Participants have developed competency in designing stable API-calling layers, handling transient failures safely, and interpreting reliability trade-offs—skills directly applicable to real-world AI backend engineering.

coursera

Key takeaways:

- Implemented a minimal reliability layer that survives rate limits and transient errors.
- Applied retry, jitter, and timeout patterns that improve p95 behavior and user experience.
- Logged structured metadata that supports debugging, optimization, and observability in production systems.

Resources

Primary Materials:

- Starter middleware script (if provided)
- Example FastAPI wrapper (optional)

Reference Documentation:

- OpenAI API reference
- Python httpx / requests exception handling docs

Additional Resource (Optional):

- Retry/backoff patterns (AWS Architecture Blog, Google SRE Book)
- Example observability patterns for microservices

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.